

Министерство науки и высшего образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №9

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Поиск расстояний в графе»

Выполнили:

студенты группы 24ВВВЗ

Задоркин Тимофей

Комиссаров Андрей

Приняли:

Юрова О.В.

Деев М.В.

Пенза 2025

Цель работы – ознакомиться с алгоритмом поиска расстояний в графе и освоить его практическую реализацию.

Общие сведения.

Поиск расстояний – довольно распространенная задача анализа графов.

Для поиска расстояний можно использовать процедуры обхода графа. Для этого при каждом переходе в новую вершину необходимо запоминать, сколько шагов до нее мы сделали. При этом вектор, который хранил информацию о посещении вершин становится вектором расстояний. Довольно просто модернизировать для поиска расстояний в графе алгоритм обхода в ширину, т.к. этот алгоритм проходит вершины по уровням удаленности, то для не ориентированного графа для вершин каждого следующего уровня глубины расстояние от исходной вершины увеличивается на 1. Удалённость в данном случае понимается как количество ребер, по которым необходимо прейти до достижения вершины.

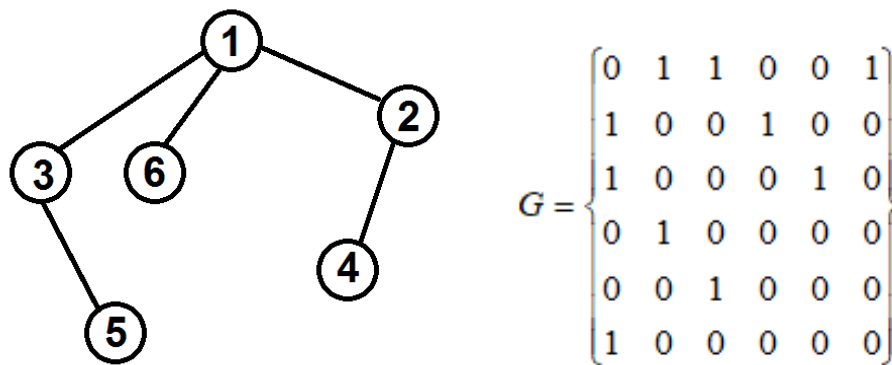


Рисунок 1 – Граф

Таким образом, можно предложить следующую реализацию алгоритма обхода в ширину.

Вход: G – матрица смежности графа, v – исходная вершина.

Выход: DIST – вектор расстояний до всех вершин от исходной.

Алгоритм ПОШ

1.1. для всех i положим $DIST[i] = -1$ пометим как "не посещенную";

1.2. **ВЫПОЛНЯТЬ** BFS(v).

1.3 для всех i вывести $DIST[i]$ на экран;

Алгоритм BFS(v):

2.1. Создать пустую очередь $Q = \{\}$;

2.2. Поместить v в очередь $Q.push(v)$;

2.3. Обновить вектор расстояний $DIST[x] = 0$;

2.4. **ПОКА** $Q \neq \emptyset$ очередь не пуста **ВЫПОЛНЯТЬ**

2.5. $v = Q.front()$ установить текущую вершину;

2.6. Удалить первый элемент из очереди $Q.pop()$;

2.7. вывести на экран v ;

2.8. **ДЛЯ** $i = 1$ **ДО** $size_G$ **ВЫПОЛНЯТЬ**

2.9. **ЕСЛИ** $G(v,i) = 1$ **И** $DIST[i] = -1$

2.10. **ТО**

2.11. Поместить i в очередь $Q.push(i)$;

2.12. Обновить вектор расстояний $DIST[i] = DIST[v] + 1$;

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.

2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При

реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.

3.* Реализуйте процедуру поиска расстояний для графа, представленного списками смежности.

Задание 2*

1. Реализуйте процедуру поиска расстояний на основе обхода в глубину.
2. Реализуйте процедуру поиска расстояний на основе обхода в глубину для графа, представленного списками смежности.
3. Оцените время работы реализаций алгоритмов поиска расстояний на основе обхода в глубину и обхода в ширину для графов разных порядков.

Результат работы программы

```
=== Поиск расстояний в графе ===
Введите количество вершин: 7

Матрица смежности:
 0  0  1  1  0  1  1
 0  0  0  0  1  1  1
 1  0  0  0  1  1  0
 1  0  0  0  0  0  0
 0  1  1  0  0  0  0
 1  1  1  0  0  0  0
 1  1  0  0  0  0  0

Списки смежности:
Вершина 0: 6 -> 5 -> 3 -> 2 -> NULL
Вершина 1: 6 -> 5 -> 4 -> NULL
Вершина 2: 5 -> 4 -> 0 -> NULL
Вершина 3: 0 -> NULL
Вершина 4: 2 -> 1 -> NULL
Вершина 5: 2 -> 1 -> 0 -> NULL
Вершина 6: 1 -> 0 -> NULL

Введите стартовую вершину: 0
```

Рисунок 1-1 – 7 вершин

=== BFS с матрицей смежности ===

Расстояния:

до вершины 0: 0

до вершины 1: 2

до вершины 2: 1

до вершины 3: 1

до вершины 4: 2

до вершины 5: 1

до вершины 6: 1

Время: 0.000028 секунд

=== BFS со списками смежности ===

Расстояния:

до вершины 0: 0

до вершины 1: 2

до вершины 2: 1

до вершины 3: 1

до вершины 4: 2

до вершины 5: 1

до вершины 6: 1

Время: 0.000010 секунд

=== DFS с матрицей смежности ===

Расстояния:

до вершины 0: 0

до вершины 1: 3

до вершины 2: 1

до вершины 3: 1

до вершины 4: 2

до вершины 5: 1

до вершины 6: 1

Время: 0.000010 секунд

=== DFS со списками смежности ===

Расстояния:

до вершины 0: 0

до вершины 1: 2

до вершины 2: 1

до вершины 3: 1

до вершины 4: 3

до вершины 5: 1

до вершины 6: 1

Время: 0.000016 секунд

Рисунок 1-2 – 7 вершин

=== СРАВНЕНИЕ ВРЕМЕНИ ВЫПОЛНЕНИЯ ===	
Алгоритм	Время (секунды)
BFSD с матрицей смежности	0.000028
BFSD со списками смежности	0.000010
DFSD с матрицей смежности	0.000010
DFSD со списками смежности	0.000016

Рисунок 1-3 – 7 вершин

```

Введите количество вершин: 10

Матрица смежности:
0 0 0 0 1 0 1 0 1 0
0 0 0 0 1 0 0 1 1 1
0 0 0 1 0 0 1 1 1 0
0 0 1 0 1 0 0 1 0 0
1 1 0 1 0 1 0 0 1 0
0 0 0 0 1 0 0 1 1 1
1 0 1 0 0 0 0 0 1 1
0 1 1 1 0 1 0 0 0 0
1 1 1 0 1 1 1 0 0 0
0 1 0 0 0 1 1 0 0 0

Списки смежности:
Вершина 0: 8 -> 6 -> 4 -> NULL
Вершина 1: 9 -> 8 -> 7 -> 4 -> NULL
Вершина 2: 8 -> 7 -> 6 -> 3 -> NULL
Вершина 3: 7 -> 4 -> 2 -> NULL
Вершина 4: 8 -> 5 -> 3 -> 1 -> 0 -> NULL
Вершина 5: 9 -> 8 -> 7 -> 4 -> NULL
Вершина 6: 9 -> 8 -> 2 -> 0 -> NULL
Вершина 7: 5 -> 3 -> 2 -> 1 -> NULL
Вершина 8: 6 -> 5 -> 4 -> 2 -> 1 -> 0 -> NULL
Вершина 9: 6 -> 5 -> 1 -> NULL

```

Рисунок 2-1 – 10 вершин

```

=== BFSД с матрицей смежности ===
Расстояния:
  до вершины 0: 0
  до вершины 1: 2
  до вершины 2: 2
  до вершины 3: 2
  до вершины 4: 1
  до вершины 5: 2
  до вершины 6: 1
  до вершины 7: 3
  до вершины 8: 1
  до вершины 9: 2
Время: 0.000035 секунд

```

```

=== BFSД со списками смежности ===
Расстояния:
  до вершины 0: 0
  до вершины 1: 2
  до вершины 2: 2
  до вершины 3: 2
  до вершины 4: 1
  до вершины 5: 2
  до вершины 6: 1
  до вершины 7: 3
  до вершины 8: 1
  до вершины 9: 2
Время: 0.000014 секунд

```

```

=== DFSD с матрицей смежности ===
Расстояния:
  до вершины 0: 0
  до вершины 1: 2
  до вершины 2: 4
  до вершины 3: 2
  до вершины 4: 1
  до вершины 5: 2
  до вершины 6: 1
  до вершины 7: 3
  до вершины 8: 1
  до вершины 9: 3
Время: 0.000007 секунд

```

```

=== DFSD со списками смежности ===
Расстояния:
  до вершины 0: 0
  до вершины 1: 2
  до вершины 2: 2
  до вершины 3: 4
  до вершины 4: 1
  до вершины 5: 2
  до вершины 6: 1
  до вершины 7: 3
  до вершины 8: 1
  до вершины 9: 3
Время: 0.000019 секунд

```

Рисунок 2-2 – 10 вершин

=== СРАВНЕНИЕ ВРЕМЕНИ ВЫПОЛНЕНИЯ ===	
Алгоритм	Время (секунды)
BFSD с матрицей смежности	0.000035
BFSD со списками смежности	0.000014
DFSD с матрицей смежности	0.000007
DFSD со списками смежности	0.000019

Рисунок 2-3 – 10 вершин

Вывод: в ходе выполнения лабораторной работы ознакомились с алгоритмом поиска расстояний в графе и освоили его практическую реализацию.

Приложение – Листинг программы

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <time.h>
#include <queue>
#include <stack>
#include <windows.h>

using namespace std;

struct Node {
    int vertex;
    Node* next;
};

void BFSD_Matrix(int** G, int numG, int* DIST, int v) {
    queue<int> q;

    DIST[v] = 0;
    q.push(v);

    while (!q.empty()) {
        int current = q.front();
        q.pop();

        for (int i = 0; i < numG; i++) {
            if (G[current][i] == 1 && DIST[i] == -1) {
                q.push(i);
                DIST[i] = DIST[current] + 1;
            }
        }
    }
}

void BFSD_AdjList(Node** adjList, int numG, int* DIST, int v) {
    queue<int> q;

    DIST[v] = 0;
    q.push(v);

    while (!q.empty()) {
        int current = q.front();
        q.pop();

        Node* node = adjList[current];
        while (node != NULL) {
            if (DIST[node->vertex] == -1) {
                q.push(node->vertex);
                DIST[node->vertex] = DIST[current] + 1;
            }
            node = node->next;
        }
    }
}
```



```

        node = node->next;
    }
}

void DFSD_Matrix(int** G, int numG, int* DIST, int v) {
    stack<int> s;
    s.push(v);
    DIST[v] = 0;

    while (!s.empty()) {
        int current = s.top();
        s.pop();

        for (int i = numG - 1; i >= 0; i--) {
            if (G[current][i] == 1 && DIST[i] == -1) {
                s.push(i);
                DIST[i] = DIST[current] + 1;
            }
        }
    }
}

void DFSD_AdjList(Node** adjList, int numG, int* DIST, int v) {
    stack<int> s;
    s.push(v);
    DIST[v] = 0;

    while (!s.empty()) {
        int current = s.top();
        s.pop();

        Node* node = adjList[current];
        stack<int> tempStack;
        while (node != NULL) {
            if (DIST[node->vertex] == -1) {
                tempStack.push(node->vertex);
            }
            node = node->next;
        }

        while (!tempStack.empty()) {
            int neighbor = tempStack.top();
            tempStack.pop();
            s.push(neighbor);
            DIST[neighbor] = DIST[current] + 1;
        }
    }
}

Node** createAdjListFromMatrix(int** matrix, int numG) {
    Node** adjList = (Node**)malloc(numG * sizeof(Node*));

```

```

    for (int i = 0; i < numG; i++) {
        adjList[i] = NULL;
        for (int j = 0; j < numG; j++) {
            if (matrix[i][j] == 1) {
                Node* newNode = (Node*)malloc(sizeof(Node));
                newNode->vertex = j;
                newNode->next = adjList[i];
                adjList[i] = newNode;
            }
        }
    }

    return adjList;
}

void printMatrix(int** G, int numG) {
    printf("\nМатрица смежности:\n");
    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d", G[i][j]);
        }
        printf("\n");
    }
}

void printAdjList(Node** adjList, int numG) {
    printf("\nСписки смежности:\n");
    for (int i = 0; i < numG; i++) {
        printf("Вершина %d: ", i);
        Node* current = adjList[i];
        while (current != NULL) {
            printf("%d -> ", current->vertex);
            current = current->next;
        }
        printf("NULL\n");
    }
}

void initDIST(int* DIST, int numG) {
    for (int i = 0; i < numG; i++) {
        DIST[i] = -1;
    }
}

void printDIST(int* DIST, int numG) {
    printf("Расстояния:\n");
    for (int i = 0; i < numG; i++) {
        printf(" до вершины %d: ", i);
        if (DIST[i] == -1) {
            printf("Недостижима\n");
        }
        else {
            printf("%d\n", DIST[i]);
        }
    }
}

```

```

    }
}

void freeAdjList(Node** adjList, int numG) {
    for (int i = 0; i < numG; i++) {
        Node* current = adjList[i];
        while (current != NULL) {
            Node* temp = current;
            current = current->next;
            free(temp);
        }
    }
    free(adjList);
}

double measureTime(void (*func)(int**, int, int*, int), int**
graph, int numG, int* DIST, int start) {
    LARGE_INTEGER frequency, start_time, end_time;
    QueryPerformanceFrequency(&frequency);
    QueryPerformanceCounter(&start_time);

    func(graph, numG, DIST, start);

    QueryPerformanceCounter(&end_time);
    return (double)(end_time.QuadPart - start_time.QuadPart) /
frequency.QuadPart;
}

double measureTimeAdjList(void (*func)(Node**, int, int*, int),
Node** adjList, int numG, int* DIST, int start) {
    LARGE_INTEGER frequency, start_time, end_time;
    QueryPerformanceFrequency(&frequency);
    QueryPerformanceCounter(&start_time);

    func(adjList, numG, DIST, start);

    QueryPerformanceCounter(&end_time);
    return (double)(end_time.QuadPart - start_time.QuadPart) /
frequency.QuadPart;
}

int main() {
    system("chcp 1251");
    int** G;
    Node** adjList;
    int numG, start;
    int* DIST;

    printf("=== Поиск расстояний в графе ===\n\n");

    printf("Введите количество вершин: ");
    scanf("%d", &numG);

```

```

DIST = (int*)malloc(numG * sizeof(int));
G = (int**)malloc(numG * sizeof(int*));
for (int i = 0; i < numG; i++)
    G[i] = (int*)malloc(numG * sizeof(int));

srand(time(NULL));

for (int i = 0; i < numG; i++) {
    for (int j = 0; j < numG; j++) {
        if (i == j) {
            G[i][j] = 0;
        }
        else {
            if (i > j) {
                G[i][j] = G[j][i];
            }
            else {
                G[i][j] = rand() % 2;
            }
        }
    }
}

printMatrix(G, numG);

adjList = createAdjListFromMatrix(G, numG);
printAdjList(adjList, numG);

printf("\nВведите стартовую вершину: ");
scanf("%d", &start);

double time_bfs_matrix, time_bfs_list, time_dfs_matrix,
time_dfs_list;

printf("\n=== BFSD с матрицей смежности ===\n");
initDIST(DIST, numG);
time_bfs_matrix = measureTime(BFSD_Matrix, G, numG, DIST,
start);
printDIST(DIST, numG);
printf("Время: %.6f секунд\n", time_bfs_matrix);

printf("\n=== BFSD со списками смежности ===\n");
initDIST(DIST, numG);
time_bfs_list = measureTimeAdjList(BFSD_AdjList, adjList,
numG, DIST, start);
printDIST(DIST, numG);
printf("Время: %.6f секунд\n", time_bfs_list);

printf("\n=== DFSD с матрицей смежности ===\n");
initDIST(DIST, numG);
time_dfs_matrix = measureTime(DFSD_Matrix, G, numG, DIST,
start);

```

```

printDIST(DIST, numG);
printf("Время: %.6f секунд\n", time_dfs_matrix);

printf("\n=== DFSD со списками смежности ===\n");
initDIST(DIST, numG);
time_dfs_list = measureTimeAdjList(DFSD_AdjList, adjList,
numG, DIST, start);
printDIST(DIST, numG);
printf("Время: %.6f секунд\n", time_dfs_list);

printf("\n=== СРАВНЕНИЕ ВРЕМЕНИ ВЫПОЛНЕНИЯ ===\n");
printf("+-----+-----+");
-+ \n");
printf("| Алгоритм | Время (секунды) | \n");
printf("+-----+-----+");
-+ \n");
printf("| BFSД с матрицей смежности | %14.6f | \n",
time_bfs_matrix);
printf("| BFSД со списками смежности | %14.6f | \n",
time_bfs_list);
printf("| DFSD с матрицей смежности | %14.6f | \n",
time_dfs_matrix);
printf("| DFSD со списками смежности | %14.6f | \n",
time_dfs_list);
printf("+-----+-----+");
-+ \n");

free(DIST);
for (int i = 0; i < numG; i++)
    free(G[i]);
free(G);
freeAdjList(adjList, numG);

_getch();
return 0;
}

```